[Seasonally Sharing] OpenClaw is not a product story, but a facet of the Agent software stack migration

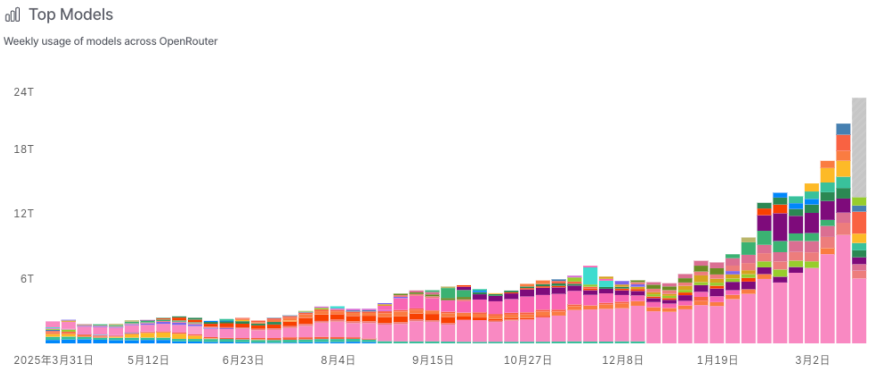
🚢 Research is continuously updating

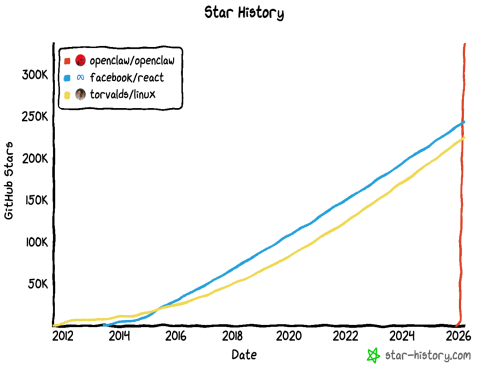
Hype: Agent Frenzy Unleashed by OpenClaw

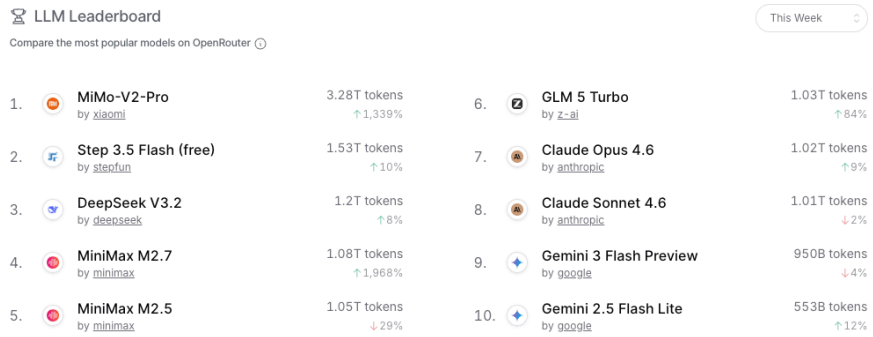
Why is OpenClaw a phenomenal breakthrough?

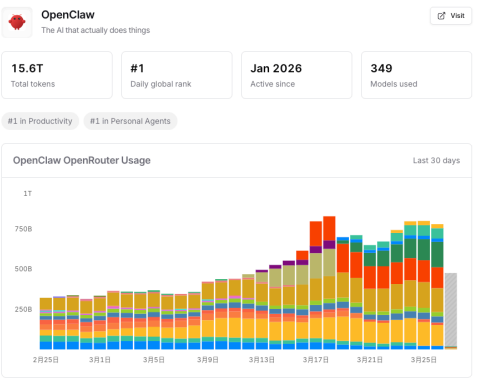
Growth Rate: Approaching the ChatGPT Moment

- Released at the end of January 2026, it took only about **60 days** to exceed **250,000** stars
- **Surpassing historical records**: It exceeded **React** (approximately 243,000 stars) and **Linux Kernel** (approximately 218,000 stars), which had been maintained for over a decade, in just two months, becoming the **fastest-growing** software project (non-resource list type) in the global history of GitHub.
    - On March 2, it surpassed Linux to become the most starred open-source project on GitHub, currently with a cumulative total of 335,000 stars, over 1,300 *contributors*, over **20,000** issues, and over 30,000 PRs.
- **Single-day peak**: During its outbreak period, it once set a record of**gaining more than 20,000 stars in a single day**.
- Usage spread to the global developer community within a very short period, with hundreds of thousands of developers participating in the ecosystem and 120,000 Discord members.
- Clawhub has accumulated **26,000 skills**
- Openrouter statistics token call top 1 application









Diffusion Path: The Promotion Speed of the First AI Native Application/Agent is Reversed Between China and the US

China: Application implementation is significantly faster (especially on the developer and industrial sides)
US: Models/fundamental capabilities are stronger, but application implementation is slower and more cautious (the US has entered the stage of security + enterprise process reconstruction)

A phenomenon that deserves great attention is:

- Chinesedevelopers and application-side implementation speed is significantly faster
  - OpenClaw usage surpasses the US
  - Chinese enterprises are deploying OpenClaw faster than their US competitors: Chinese enterprises are integrating OpenClaw with domestic AI models, the costs of which are 60-80% lower than their Western counterparts
    - **Kimi**: First claimed that the model supports OpenClaw (**k2.5** officially released on January 27); Launched the cloud-hosted version of **Kimi Claw** in 2.15, opened the Beta test to paid members, and pre-installed the ClawHub skill library.
    - **MiniMax**: Cloud-hosted version of **MaxClaw** to be launched at the end of February, based on **M2.5** model (leading in speed and cost, released on February 12), integrated into MiniMax Agent web client.

        - **Zhipu** : (**GLM-5** officially released on February 13th) On March 10th, the local packaged version of **AutoClaw (Aolong)** was launched, the first local version of OpenClaw installed with one click in China, with more than 50 popular skills preset, supporting one-click access to instant message tools such as Feishu; On March 16th, **GLM-5-Turbo** was launched, which was deeply optimized for the needs of OpenClaw in tool invocation and long-link execution.
        - **Step** : On March 5th, the model **Step 3.5 Flash** (released on March 4th) increased by more than 20 times compared to 30 days ago, and the two domestic large models, MiniMax M2.5, accounted for about 50% of the total token consumption of the OpenRouter platform in a single day.
    - Just six months ago, US companies still dominated the early applications of OpenClaw, with startups and enterprises like Cognition AI testing**agent frameworks for code assistance and workflow automation**. However, the support from the Chinese government, the highly competitive prices of domestic AI providers, and the urgent demand for automation from a large number of enterprises have jointly created the perfect conditions for the explosive growth of OpenClaw.
- **Popular among the public, even a bitwildly growing**
    - Users queue up for installation and find someone to deploy on their behalf
    - WeChat directly transforms into Agent UI (Universal Entry)
    - Robot, IoT, and Manufacturing Access
- **Multiple local governments are promoting AI individual entrepreneurship and have introduced the OpenClaw support policy: computing power subsidies + scenario opening + startup funds**
    - On March 6, Futian District, Shenzhen, took the lead in releasing AI Digital Intelligence Employee 2.0, deploying the "Government Lobster" intelligent agent, focusing on reducing the burden and increasing efficiency at the grassroots level, and promoting the innovative application of OpenClaw in the government affairs field.
    - On March 8, Longgang District, Shenzhen City, released the "Several Measures to Support the Development of OpenClaw & OPC (Draft for Comment)" (abbreviated as the "Ten Measures for Lobster"), clearly proposing support measures such as encouraging the launch of "Lobster Service Areas", providing subsidies of up to 2 million yuan for OpenClaw-related skill development and application projects, and selecting demonstration projects to award up to 1 million yuan in incentives, to help the OpenClaw ecosystem take root;

- **Hefei High-tech Zone (March 6)** : Released "15 Guidelines for Lobster Farming", launched "Computing Power Voucher + Corpus Voucher + Model Voucher", and provided up to **10 million yuan in financial support** for AI startup projects, focusing on building the "One-Person Company (OPC)" ecosystem.
- **Wuxi High-tech Zone, Jiangsu (early March)**: Launched "12 Measures for Cultivating OpenClaw", with supporting industrial funds (approximately 3 billion yuan) and open application scenarios, offering a maximum subsidy of approximately **5 million yuan** per project to promote the implementation of AI + manufacturing.
- **Changshu, Jiangsu (March 9)**: Approximately 13 measures were announced to build an OPC community by integrating local industries such as textiles and e-commerce, with subsidies for some projects reaching up to approximately **6 million yuan**.
- **Nanjing Qixia High-tech Zone, Hangzhou Xiaoshan District and other places** : have also followed up to introduce or prepare support policies related to "lobster", forming a regional competition situation
- March 10: Security Alert, the National Internet Emergency Center issued a "Risk Alert on OpenClaw Security Application" + subsequent series of security alerts and restrictions on state-owned enterprises and government agencies from running OpenClaw applications on work hardware, citing significant cybersecurity risks.

Acceleration of Industry Collaboration Driven by Ecosystem Openness

Within a few weeks after OpenClaw:

- **Model Vendors: Launch or Strengthen Agent-Oriented Models**
  - A typical example is GLM-5-Turbo, which is directly optimized for OpenClaw scenarios, with key capabilities including: Tool Use, Planning, Long-horizon execution, and Persistent tasks
  - Optimized Models: Step-3.5 Flash, MiniMax 2.x, etc., optimized for long-chain tasks and tool invocation
- **Cloud and Infrastructure Vendors: Launch Agent runtime / sandbox / control layer**
  - For example, Nvidia has launched NemoClaw + OpenShell, providing governance and runtime capabilities
- Large enterprises and AI startups quickly complete entry-level access and ecosystem binding, with the platform opening up API / MCP / mini-program access
  - **Entry Layer: The enterprise collaboration platform becomes the default interaction interface for Agent**

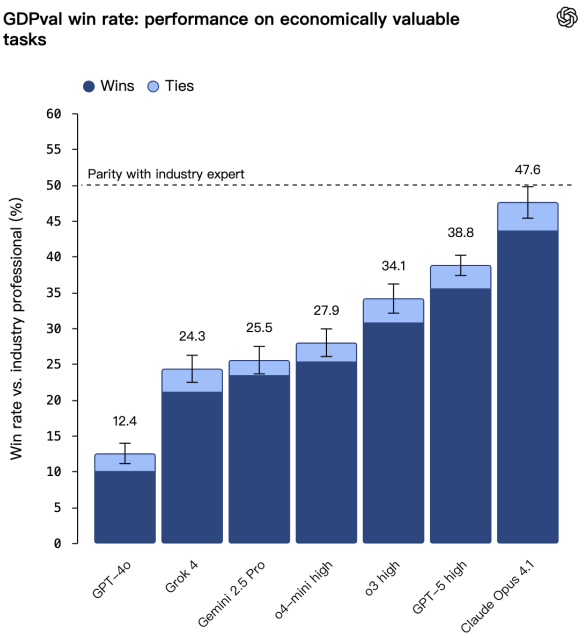
        - Feishu, DingTalk, WeCom, etc. are fully integrated; supports directly invoking Agents via chat to execute tasks (such as document processing, automated workflows); unified access to multiple IMs has become a standard capability
    - **Interface layer: The platform opens APIs / Skills / MCP to form a standardized call system**
        - Supports Skills, plugins, and MCP protocol; Agents can directly call applications; Unified orchestration of multiple models, multiple tools, and multiple applications
    - **Product layer: Large enterprises and startups are fully Claw-ized, forming multi-form competition**
        - Cloud/Platform Vendors: Alibaba: CoPaw, HiClaw (local + cloud, supporting Feishu/DingTalk operations); ByteDance: ArkClaw (SaaS, deeply adapted to Feishu); Tencent: QClaw / WorkBuddy (enterprise + WeChat entry)
        - Model Vendors / Startups: Zhipu AutoClaw, Kimi Claw, MiniMax MaxClaw, etc.
        - Terminal/OS Manufacturer: Xiaomi Miclaw (System-level Agent)

Underlying Drivers

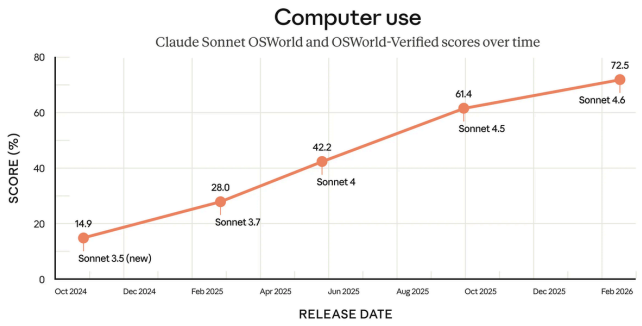
Advances in Cutting-Edge Model Capabilities

- **First, the model begins to possess the unified capabilities of tool use, browser use, and computer use: from generation to perceiving the screen and performing operations**

时间节点	代表模型	OSWorld
2024.04	GPT-4o / Claude 3 Opus	
2025.07	Claude 3.5 / Gemini 2	~40% - 47 Verified，
2025.12	Claude Sonnet 4.5	
2026.1	Kimi K2.5	
2026.02	Claude Sonnet 4.6	72.5%（I
2026.03	GPT-5.4	



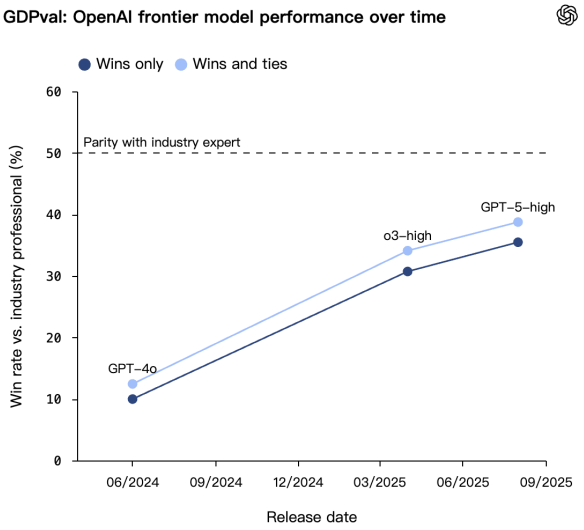
https://openai.com/index/gdpval/



https://www.anthropic.com/news/claude-sonnet-4-6

Figure 1: OSworld bench: Measures the comprehensive ability of AI to perform complex tasks on a computer like a human, including dimensions such as cross-application task understanding and planning, real GUI interaction, MultiModal Machine Learning perception, long-sequence execution and feedback adjustment, general tool usage, and end-to-end task completion ability.

- From the release of this evaluation in 2024, it took less than two years for the initial cutting-edge model score of only 12.2% to reach the point where Claude Sonnet 4.6 officially surpassed the human expert benchmark (72.4%).
- GPT 5.4 is also OpenAI's first native computer-use model and is currently the SOTA in this evaluation



https://openai.com/index/gdpval/

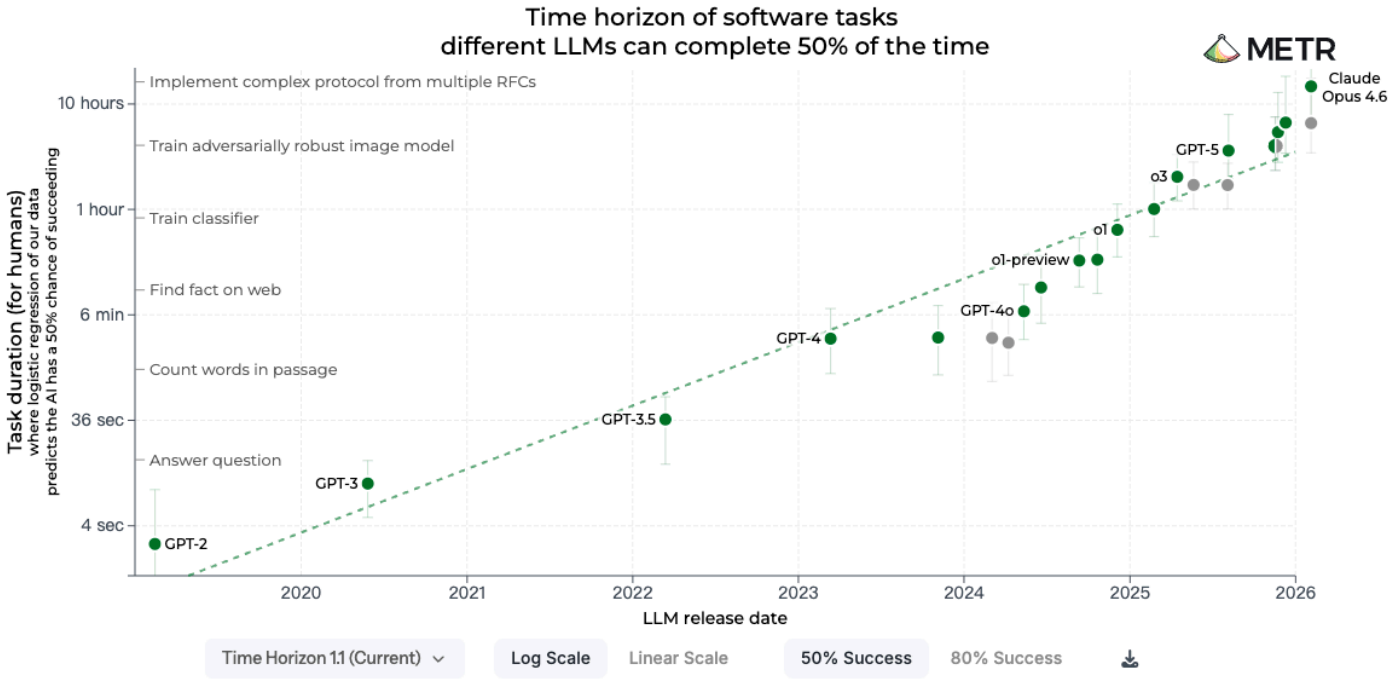
Figure 1: GDPval: Measures the model's performance in real-world tasks with economic value, covering 44 selected professional fields from the top nine industries of the US GDP, including customer service, financial investment, law, etc.

Cutting-edge models have approached the work quality of Subject-Matter Experts, with the speed of cutting-edge models completing the GDPval task being approximately 100 times that of Subject-Matter Experts, and the cost being only one-hundredth of theirs.

Figure 2: From GPT-4o to GPT-5, the model's performance on the GDPval task increased by more than threefold within a year.

- **Second, the model is not only capable of performing a single step but can also work continuously for a longer period of time.**
  - Anthropic's research on real agent usage shows that in the longest running session, the duration of continuous autonomous work of Claude Code increased from less than 25 minutes to over 45 minutes within three months. This is not a story of "model IQ improvement", but rather**an improvement in continuous execution ability**. Anthropic
  - METR: The duration of tasks autonomously completed by Claude Opus4.6 AI Agent with 50% reliability has reached 14.5 hours

- METR measures the performance of artificial intelligence by the duration of tasks that an AI Agent can complete autonomously. Over the past six years, the length of tasks that a general-purpose cutting-edge model Agent can complete autonomously with 50% reliability (measured by the time required for professionals to complete these tasks) has approximately doubled every seven months. By inference, within less than a decade, we will see AI Agents capable of independently completing most software tasks that currently take humans days or even weeks to complete.



https://metr.org/blog/2025-03-19-measuring-ai-ability-to-complete-long-tasks/

Leading model companies are all focused on refining new developer work stacks: System/Runtime capabilities are gradually being completed (long-term operation + execution)

"Tool + Protocol + Framework"

Protocol Layer - MCP Transition from Single Company Standard to Industry-wide Standard

Framework Layer - Major AI companies have launched open-source SDKs (OpenAI Agents SDK, Claude Agent SDK, Google SDK)

Developer toolchains: OpenAI acquired Astral, Anthropic acquired Bun, and Google DeepMind acquired the Antigravity team.

	Framework	Runtime	Harness
Package	LangChain	LangGraph	Deep Agents
Value add	Abstractions	Durable execution, streaming, HITL, persistence	Predefined tools, prompts, subagents
When to use	- Want to get started quickly - Want to standardize how team builds	- Want low level control - Long running, stateful agents	- More autonomous agents
Other Options	AI SDK, LlamaIndex, CrewAI, Google SDK, OpenAI Agents SDK	Temporal, Inngest	Claude Agent SDK

https://blog.langchain.com/agent-frameworks-runtimes-and-harnesses-oh-my/

Optimization of the model and Infra layer for application and inference scenarios: Deployability

- **Model Architecture Optimization (Reducing Unit Inference Cost)**
  - The Mixture of Experts (MoE) model with sparse activation decouples the parameter scale from the inference cost
  - Attention mechanism improvements (e.g., MLA / GQA), compressing KV cache, and reducing memory and bandwidth overhead
  - Linear complexity architectures (Mamba, Kimi Linear, etc.) reduce the inference complexity from $O(n^2)$ to $O(n)$, improving the efficiency of long sequences
- **Model Compression and Edge Deployment (Enhancing Usability)**
  - Techniques such as quantization, pruning, and distillation reduce model size without significantly sacrificing performance
  - Promote the extension of models to edge-side and local deployment
- **Long Context and Inference Optimization (Adapted to Agent Scenarios)**
  - Long context ability has become a key capability, but it brings about the issue of KV cache growth
  - Improve long task execution capabilities through KV compression and structural optimization
- **Inference Framework and System-Level Optimization (Improving Throughput and Stability)**
  - KV Cache Management and Compression
  - Decoding optimization (speculative decoding, etc.)
  - Optimization of inference frameworks (such as vLLM) and scheduling mechanisms

- **Infra (Infrastructure) Layer**

- **Software-hardware collaboration (Algorithm + System + Hardware) has become a key methodology: through cross-layer collaborative design, the optimal balance between "efficiency and accuracy" is achieved under the constraints of computing power and bandwidth. A typical example is** the model structure of DeepSeek, which has been "tailor-made" optimized (optimized through hardware-aware collaborative design) to a large extent for the architecture and cluster topology of the H800 chip
- **Specialized evolution for inference workloads has emerged at the chip level**:
  - GPU evolves towards inference optimization (e.g., H100, Blackwell), enhancing low-precision computation (FP4/FP8) and decoding performance
  - Dedicated accelerators such as TPU and NPU are optimized for low-latency and low-power scenarios, and replace general-purpose GPUs in some inference scenarios
  - The CPU is re-emphasized in the Agent system, taking on the roles of scheduling and execution, and forming a CPU+GPU collaborative architecture
  - Collaborate with computing and networking (such as high-performance interconnects) to continuously reduce the inference cost per token
- **Advanced packaging and high-bandwidth interconnection have become the key directions for bottleneck breakthrough:** Through 2.5D/3D packaging and high-speed interconnection, improve the multi-chip collaboration efficiency, optimize the data flow overhead of KV cache, etc., and reduce the energy consumption of data transfer.

Open Ecosystem: Protocols, Skills, Open Source Models

- The first layer is **protocol openness** . MCP

关键数据	简要说明	来源网址
MCP 完成中立标准化治理	2025 年 12 月，MCP 正式捐赠给 Linux 基金会旗下 Agentic AI Foundation，由 Anthropic、OpenAI、Google、Microsoft、Amazon 等全球巨头共同支持，成为厂商中立的行业开放标准	https://m.toutiao.com/group/7621397347167191604/
MCP SDK 月下载量突破 9700 万次	截至 2026 年 2 月，MCP 官方 Python/TypeScript SDK 月下载量达 9700 万次，开发者采纳度持续攀升	https://m.toutiao.com/group/7621397347167191604/
全球公开 MCP Server 超 17000 个	截至 2026 年 1 月，全球已上线超 17000 个公开可用的 MCP Server，覆盖代码、办公、云服务等全场景	https://zuplo.com/mcp-report
开发者增长预期明确	72% 的 MCP 技术从业者预计未来 12 个月内 MCP 使用量将持续增长，54% 的从业者认可其长期行业标准地位	https://zuplo.com/mcp-report
行业渗透率预测	Gartner 预测，2026 年 75% 的 API 网关厂商、50% 的 iPaaS 厂商将原生支持 MCP 协议	https://aicoding.juejin.cn/post/7615161431983423498

- Layer 2 **skills and plugin ecosystem are open.**

关键数据	简要说明	来源网址
ClawHub 技能总量突破 30000 个	截至 2026 年 3 月下旬，OpenClaw 官方技能市场 ClawHub 注册 Skill 数量突破 30000 个，日均新增 150 个以上，总下载量突破 1340 万，覆盖 31 个主流场景分类，全面兼容 MCP 协议	https://blog.csdn.net/ailuloo/article/det
技能开发门槛大幅降低	58% 的 MCP 技能开发者基于现有 API 封装 MCP 服务；主流开发框架 FastMCP 使用率达 42%，Anthropic 官方 SDK 使用率达 38%，仅 20% 开发者需从零构建 MCP 服务	https://zuplo.com/mcp-report
国内生态闭环落地	2026 年 3 月，阿里云魔搭社区正式上线 Skills 中心，全面对接 MCP 协议与平台内开源模型，实现「协议 - 技能 - 模型」一站式开发与调用闭环	https://developer.aliyun.com/article/171
行业标准共识形成	Anthropic 官方 Skills 开源仓库上线 4 个月内 GitHub Star 数超 62000 个，与 MCP 协议形成互补，定义了行业通用的技能开发标准	行业公开技术社区数据

- The third layer is open source models such as Qwen, DeepSeek, GLM, etc.

关键数据	简要说明	来源网址
全球开源模型已成为产业落地核心底座	2026 年 Q1，全球生成式 AI 商用落地项目中，采用开源模型的占比达 68.2%，开源模型是开放生态的核心推理与算力底座	https://www.idc.com/cn/zh/r report
全球开源模型生态规模持续爆发	截至 2026 年 3 月，Hugging Face 平台累计收录开源大模型超 45 万个，2025-2026 年新增量同比增长 127%，模型累计下载量突破 1200 亿次，全球开发者贡献者超 280 万人	https://huggingface.co/blog/s
国内主流开源模型占据绝对市场主导	2026 年国内开源大模型市场中，Qwen、DeepSeek、GLM 三大核心系列合计占据 72% 的市场份额，是国内开源生态的主流底座，覆盖 90% 以上的企业级私有化部署场景	https://www.caict.cn/kxyj/yjb
主流开源模型普遍具备完全开放属性	全球下载量 TOP100 的开源大模型中，94% 采用 Apache 2.0、MIT 等宽松商用开源协议，无商用门槛，支持无限制二次开发与分发；Qwen、DeepSeek、GLM 均在此列，完全符合开放生态核心要求	https://huggingface.co/blog/s
开源模型对开放生态的全链路适配度超 90%	截至 2026 年 3 月，魔搭 ModelScope 平台内 92% 的主流开源模型（含 Qwen、DeepSeek、GLM 全系列）已完成 MCP 协议原生适配，可无缝对接平台开放 Skills 中心，实现「协议 - 技能 - 模型」三层架构全链路打通	https://developer.aliyun.com/
主流开源模型普遍具备支撑上层生态的核心技术能力	2026 年 LMSYS 全球大模型工具调用评测榜单显示，全球 TOP20 开源模型的工具调用平均准确率达 91.3%，Qwen、DeepSeek、GLM 系列均进入全球 TOP10，原生兼容标准化 Skills 开发规范	https://arena.lmsys.org/leade
中国开源模型全球影响力登顶	2026 年春季 Hugging Face 报告显示，中国开源模型全球下载量占比达 41%，首次超越美国；Qwen、DeepSeek、GLM 为核心贡献者，是全球开放模型生态的核心组成部分	http://m.toutiao.com/group/

The Sensory Gap Behind the Hype: Why Do Many People Respond with "So What?" After Installing OpenClaw ?

Agent narrative has entered the second phase, but the product experience remains in the first phase. The first phase is to prove that it can be done; the second phase is to prove that it is worth doing, sustainable, and reassuring to do.

Token Cost: The cost per effective task is too high

Status Quo

AI agents like OpenClaw, which can "get things done", need to break down a complex instruction into dozens of steps, with each step requiring the invocation of a large model for decision-making and execution. This results in a Token consumption that is 100 to 1,000 times that of traditional chat AIs, and a complex task may consume tens of thousands or even hundreds of millions of Tokens. A single complete control loop of OpenClaw can consume hundreds to thousands of tokens in a single invocation, and the monthly cost for high-frequency scenarios on a single device can reach several thousand yuan.

Cost Composition

Why? Each agent loop in OpenClaw includes: receiving messages → context assembly → model inference → tool execution → streaming response → persistence.
https://docs.openclaw.ai/concepts/agent-loop?utm_source=chatgpt.com

Agent Runtime will **assemble context from session history and memory** in each round, and then call the model, which means reloading historical messages, system prompts (agent.md, soul.md, skill.md), memory, tool definitions / results, and the current
input.ppaolo.substack.com

Tokens per step (input + output), number of steps (number of agent loop iterations), number of workflows (the complete process of a series of model calls + tool operations to achieve a goal) - the number of steps × input tokens will grow explosively, especially in browser web scraping, repeated long context passing, and agent self-reflection loops

消耗构成项	普通聊天大模型（ChatGPT/豆包）	OpenClaw默认配置	消耗差异倍数
系统提示词	100-500 Token	1000-5000 Token	10倍+
工具调用内容	无（纯对话无工具）	5000-20000 Token/次	无上限
多代理上下文重复	无	2-5倍重复传递	2-5倍
对话历史记忆	可选自动截断	默认全量永久保留	指数级上涨
失败重复扣费	仅1次调用扣费	默认3次重试+多模型兜底	3-5倍

- **Core Expenses: Model API calls, which account for the largest proportion and are the most volatile.**
  - As the number and complexity of automated tasks increase, costs do not grow linearly but may instead multiply. The more automated tasks there are, the higher the API trigger frequency (per heartbeat), and the higher the costs. Meanwhile, **the retry loop after failure. "During 7x24h operation, the losses caused by the retry loop 'resulted in a loss of $120 overnight due to the retry loop'."**
  - **API Trigger Frequency:** Each workflow trigger, each multi-step process, and each tool invocation will generate an independent API request.
  - Token Cost Estimation: Number of Triggers (number of daily task launches) × Number of Workflow Steps × Token Scale (input, output) × Parallelism (multi-agent)
  - High Premium Model:
    - **Browser Automation**(Although OpenClaw can reduce token usage by approximately 90% by parsing the accessibility tree instead of sending screenshots, navigation still requires repeated model decisions.)
    - **Parallel task execution**
    - **Batch document processing**.
    - **Multi-agent collaboration:** Communication between multiple Agents will exponentially increase the number of requests.
    - **Large-scale long context retrieval**
- Hosting costs (on-demand): If there is no always-on device, deployment in a cloud environment is required, which involves hosting costs, but these account for a very small proportion of the total cost

- Other related costs: data backup, storage space, monitoring tools, macmini hardware devices, etc.

🥇 input = system prompt + historical messages (growing larger) + tool returns (possibly huge) + agent state

- **System Prompt, repeated with each request**: Core character settings, list of available tools, security restriction rules, user preference settings. Typically runs **5k to 10k tokens**.
  - Although Anthropic provides prompt caching (cache hits are only charged 10% of the fee), the cache has a TTL (Time To Live) limit
- **Continuously accumulated context**: Each time a new request is sent,**OpenClaw will send the entire conversation history to the AI model.**
  - Even simple problems require processing more than 200,000 cached context tokens; hundreds of thousands of `cacheRead` tokens per interaction; the longer the session, the more exponential the cost growth.
  - System prompt, historical messages, tool calls/results, attachments, and compaction are all counted towards the context
- **Tool Storage**: OpenClaw stores all tool call outputs in the session log
  - JSON and logs returned by the tool: For each chat session, all historical messages will be saved in the form of a JSONL file in the `.openclaw/agents.main/sessions/` directory. Log traversal and export will consume tens of thousands of tokens
- **Multi-round inference (multiple model calls in a single task)**
- **Heartbeat and Background Tasks**
  - Each heartbeat trigger is a complete API call. If misconfigured, it may trigger every few minutes. Each trigger contains the complete session context.
- **Retry on failure**

Conservative estimate of cost

用户类型	典型月度 token 量级	典型 token 费用	以 M2.7 模型测算	使用
轻度个人	5M – 20M	约 1–10 美元（有的文档给出 10–30 美元，取决于模型单价）sentisight+1	3-12 美元	偶尔
中等工作流	20M – 50M	约 10–70 美元。sentisight+1 $30	12-30 美元	固定用。
重度自动化	50M – 200M	约 70–150 美元。apiyi+1	30-120 美元	24/7
极端案例	200M+	可达到 2000+ 美元（用最贵模型、连续跑）。apiyi		博主

Note: The minimum cost refers to the MiniMax M2.7 model, which is currently considered the most cost-effective by the community; the input:output ratio is assumed to be 2:1 (in the OpenClaw scenario, since system prompt, historical messages, tool calls/results, attachments, and compaction are all counted towards the context, the input tokens will be significantly higher than the output tokens in many actual tasks).

Nature: the problem currently faced by all agents

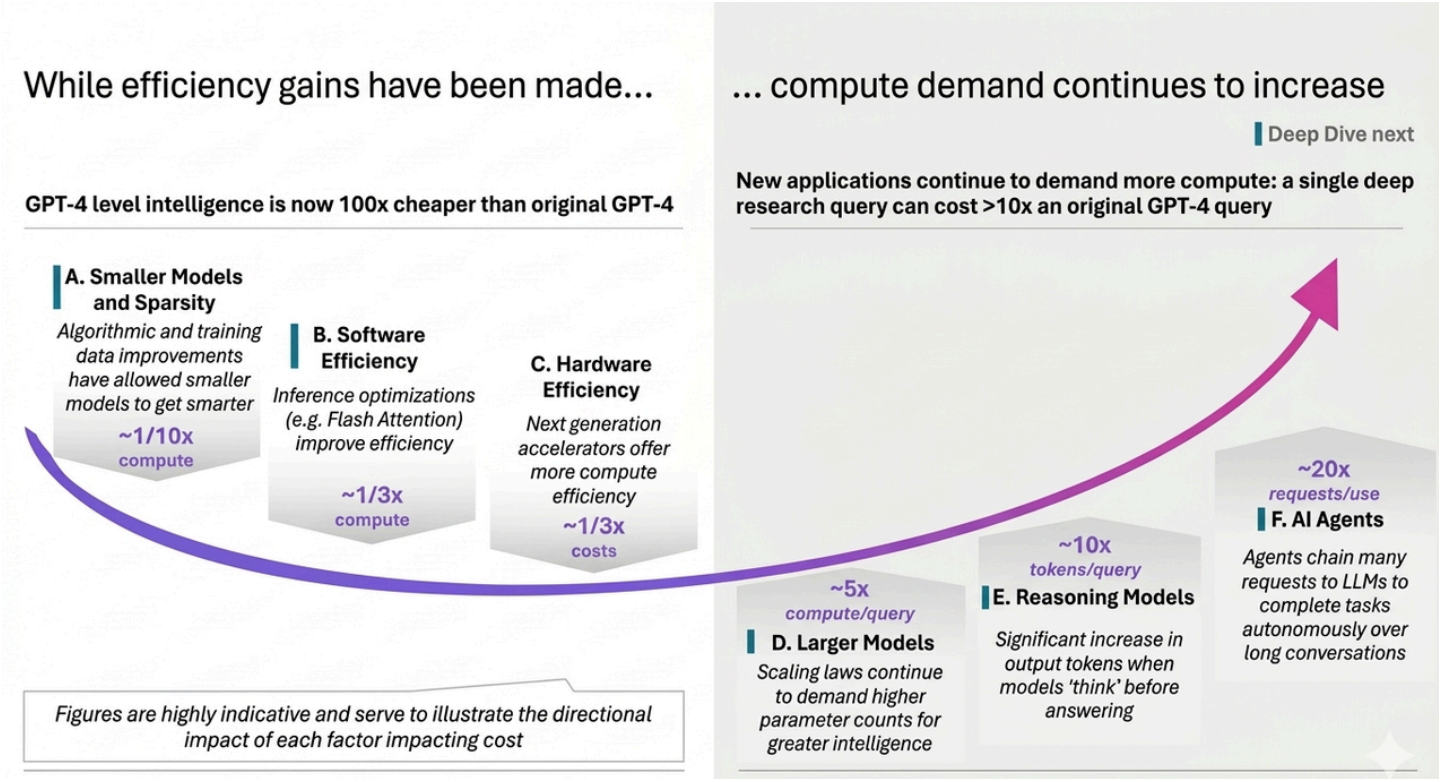
When AI starts **executing tasks** , system costs no longer consist solely of model calls, but also include state management, memory, tool calls, multi-round planning, and context maintenance. OpenClaw is just the first project to expose this issue.

Evidence 1: **Artificial analysis - the "smiling curve" of AI costs: intelligent cost decreases -, inference/application cost increases ++**

- **On the left, the unit cost of GPT-4 level intelligence has decreased by 100–1000 times (this is brought about by small models, inference efficiency, and hardware advancements); on the right, the inference cost is essentially a multiplier effect. On the one hand, although small models can achieve the intelligence level of past large models, we still hope to use larger and more cutting-edge models to perform more complex tasks. Meanwhile, when placed in the agent workflow, this cost increases exponentially.**Agentic workflow + cutting-edge inference model → **Token × rounds × tool calls = new cost peak**

Unit intelligence has become cheaper, but since we have started pursuing **more complex and in-depth applications** , the demand for inference computing power and costs are still rising. Agent inference costs are at least 20 times the initial cost of gpt-4 conversation queries. Agents need to chain multiple requests to call APIs in long conversations to autonomously complete tasks. This is because the inference chain is long and the number of requests is large; they need to maintain memory and state in long conversations, and as the task progresses, the context

(Context Window) carried by each request will become larger and larger; Autonomous correction, self-checking; if incorrect, it will re-run the previous steps.



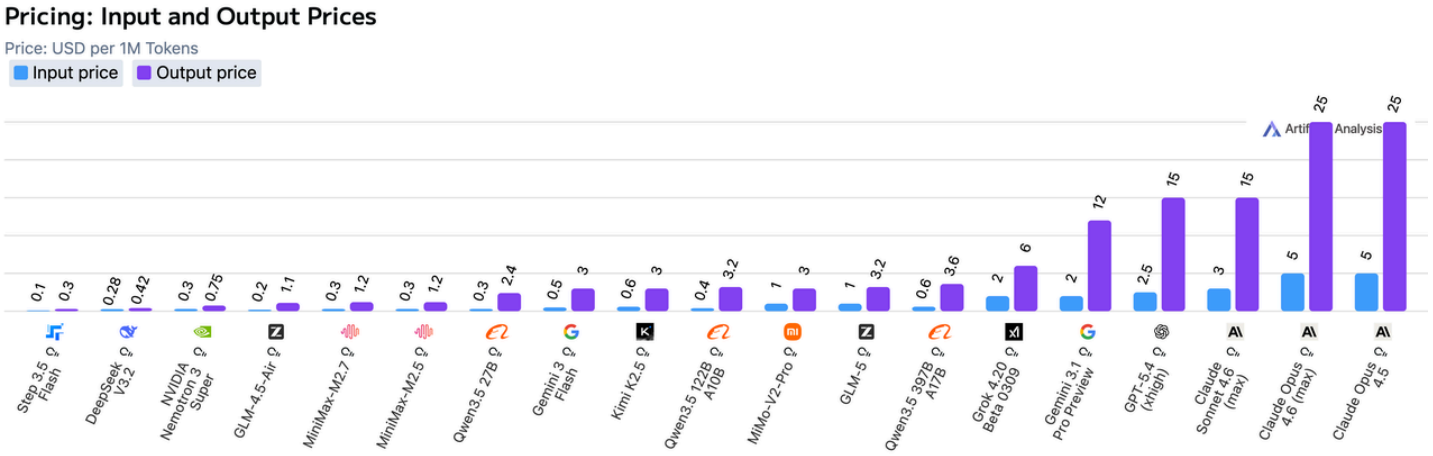
数据来源：Artificial Analysis

Evidence 2: Since an Agent is not generated in one go, but through multiple rounds of reasoning, trial and error, tool invocation, result retrieval, and then further reasoning. Google clearly states that both the intermediate reasoning tokens and tool result backfilling in the agentic loop are subject to charges; OpenAI also separately charges for essential agent capabilities such as web search and containers. This means that many "seemingly automated" actions may result in a long billing chain in the background. Google AI for Developers openai.com

The autonomous chained requests in long conversations can be broken down into the following key mechanisms:

- The Reasoning Loop: The Agent does not directly provide the final answer but follows a pattern similar to **ReAct** (Reasoning + Acting):
  - **Thought:** Based on the current task and conversation history, analyze what to do next.
  - **Action:** Select and invoke a tool (e.g., search the web, run code, query a database).
  - **Observation:** Receive the results returned by the tool and feed them back to the model.

    - **Repeat:** The model starts the next round of thinking based on newly observed information.
- State and Persistence: In long conversations, agents need to remember previous decisions.
    - **Conversation Memory:** Records all Prompts, LLM responses, and tool outputs.
    - **Context Compacting:** When a conversation becomes too long and exceeds the model's Token limit, **Harness** will be responsible for summarizing old information or pruning secondary details to ensure that the core objectives are not lost.
- Chain reaction of tool scheduling: A complex task (e.g., "research AI trends in 2024 and write a report") will be broken down into dozens of sub-requests:
    a. **Request 1:** Search for keywords.
    b. **Request 2:** Click on high-value links and extract content.
    c. **Request 3:** Information is found to be insufficient, so a re-search for specific details is conducted.
    d. **Request 4:** Consolidate all information and start writing.
- Where does autonomy manifest itself?
    - Ordinary LLMs are merely "repeaters" or "predictors", while agents possess**closed-loop control**: if a tool execution reports an error (e.g., Python code fails to run), the agent will see the error message,**independently decide**to modify the code and rerun it, rather than directly reporting the error to the user.



PinchBench 调用成功率top20 的模型 API 成本

0.1 USD to 5 USD; 0.3 USD to 25 USD

0.3 USD; 1.2 USD

Lu Bangxing: The biggest bottleneck is not the inability to do something, but the inability to do it steadily

Problems Encountered

Problems such as AI hallucinations, unstable task execution, and imperfect self-correction logic occur frequently, failing to meet users' core requirements for system stability.

- Despite the presence of `EXECUTE LOCK` and heartbeat rules and other protective measures, the core behavior still has **the problem of agent execution failure**
  - ***The format of the tool call result is incorrect, and the model doesn't know how to proceed*** *: During multi-step task execution, if a tool call fails or returns an abnormal result, OpenClaw may not be able to handle the error correctly, leading to task interruption or inconsistent states. For example, in the automatic invoice organization task, if PDF parsing fails, it may trigger a chain of errors in subsequent steps and even result in accidental data deletion.*
  - ***When the context is too long, the model starts to "hallucinate memories"*** *— treating events that have not occurred as if they have: OpenClaw's persistent memory mechanism (stored in Markdown files) may experience issues such as inconsistent states or memory pollution in cross-session or complex task scenarios. For example, if a user modifies the parameters of the same task in different sessions, it may cause OpenClaw to confuse states and make incorrect decisions.*
  - ***Task Execution Interruption and Stagnation****: In a multi-step task, if one step fails midway and there is no retry logic, the entire chain is broken; complex tasks may still result in an infinite loop due to insufficient inference verification*
  - **Scheduled tasks are unreliable** : Tasks fail in over 50 edge cases and have no error correction capabilities.
  - **Third-party dependency failure**: Application updates often cause the entire workflow to interrupt.
- **Runtime crashes and resource issues: The Gateway process** frequently hangs or crashes under load. Common triggers include:
  - **Zombie processes are occupying ports** (e.g., port 18789), which need to be manually cleaned up — in extreme cases, users reported needing to restart more than 47 times.
  - OOM (Out of Memory) forced shutdown caused by memory-intensive scheduled tasks (Cron), memory indexing, or multi-channel concurrency**.**
  - **Connection timeout** (15 seconds SSE stream) interrupted long-running tasks such as browser automation.

- **Environment mismatch**: Node.js version conflict (22+ required), missing pnpm or Xcode tools, resulting in installation/startup failure.

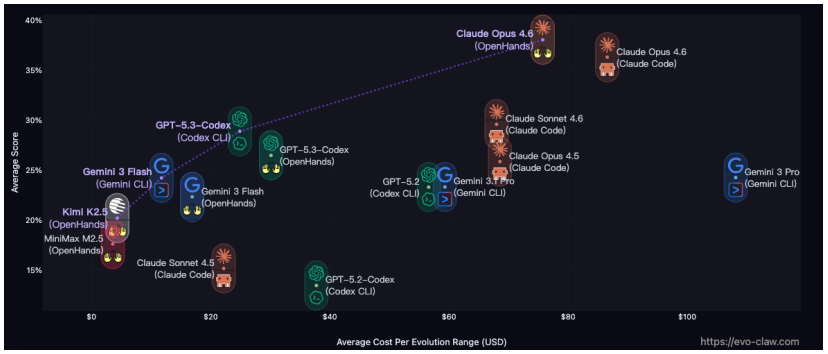
Reason

In traditional software, uncertainty mainly lies in the development phase. After developers have clearly written the logic, the system executes according to established rules, and the results are usually predictable; even if problems occur, they can be located and fixed through debugging, testing, and rolling back.

The Agent system is quite different. It hands over a portion of the processes that were originally hard-coded by developers in advance to the runtime model for decision-making. Instead of simply executing fixed instructions, the system dynamically generates the next action based on the current context, tool availability, and environmental feedback.

As a result, uncertainty enters the execution process itself: how each step is taken, which path is followed, and what the final outcome will be may all change with environmental changes, and are also more difficult to fully predict in advance.

- OpenClaw is still in the experimental autonomous agent framework stage, and moreover, the capabilities of the entire Agent for complex long-range tasks need to be improved.
- **EvoClaw: Evaluating the performance of AI Agents in continuous software evolution. Experimental results show that current AI Agents do not yet possess the qualities of a true "software engineer".** They excel at solving local, immediate programming problems, but **lack the ability to manage complexity, maintain system consistency, and govern the codebase over the long term in a dynamically evolving environment.** Architectures for "long-term memory" and "system-level verification" are needed.
    - When an Agent needs to continuously complete multiple milestone tasks on the same codebase, the success rate experiences a cliff-like decline. The average success rate of state-of-the-art Agents in this mode is only **38%** , and even drops to **13.37%** in the most complex long-sequence tasks.



#	Model	Agent	Score (%)	Precision (%)	Recall (%)	Resolve (%)	Cost ($)	Out Tok. (K)	Time (h)	Turns
1	Claude Opus 4.6	OpenHands	38.03	37.33	55.21	8.46	75.73	524	7.54	1,970
2	Claude Opus 4.6	Claude Code	36.29	37.84	56.32	11.57	86.56	578	2.73	1,891
3	Claude Sonnet 4.6	Claude Code	29.58	29.62	47.63	5.88	67.98	852	4.41	1,538
4	GPT 5.3 Codex	Codex CLI	28.88	27.81	49.70	9.58	24.88	392	8.23	1,109
5	GPT 5.3 Codex	OpenHands	26.47	25.01	37.32	12.50	30.13	553	18.75	1,047
6	Claude Opus 4.5	Claude Code	25.85	28.04	40.80	6.28	68.55	309	2.35	999
7	Gemini 3 Pro	Gemini CLI	24.25	25.46	32.70	13.37	108.12	294	3.65	676
8	Gemini 3 Flash	Gemini CLI	24.22	24.31	42.12	8.37	11.75	255	5.02	1,512
9	Gemini 3.1 Pro	Gemini CLI	23.32	21.59	37.22	10.95	59.31	207	3.89	1,208
10	GPT 5.2	Codex CLI	23.30	20.89	45.76	8.18	56.59	814	9.35	1,717
11	Gemini 3 Flash	OpenHands	22.32	25.20	37.13	6.59	16.90	1,516	7.16	2,632
12	Kimi K2.5	OpenHands	20.20	26.29	31.37	8.49	4.32	279	6.85	800
13	MiniMax M2.5	OpenHands	17.60	22.48	34.60	1.30	3.57	598	11.88	1,846
14	Claude Sonnet 4.5	Claude Code	15.16	18.88	28.50	5.49	22.18	243	2.06	770
15	GPT 5.2 Codex	Codex CLI	13.46	12.65	26.65	4.78	37.74	701	3.56	1,259

https://evo-claw.com/

Lack of applied imagination: It's not that users lack imagination, but rather that the industry has yet to find truly high-value, high-tolerance entry points

The ClawHub platform has already aggregated over 21,000 skill plugins, covering scenarios such as browser automation, email management, scheduling, file processing, code writing, and smart home control. However, actual user usage is mainly concentrated in several high-frequency scenarios: **message automation, email automation, web search and information scraping, file organization and classification, code-assisted writing, and schedule management and reminders.**

- Message Automation (45%): Consolidated statistics for skills related to WhatsApp, Telegram, Slack, and Email
- Code Workflow (28%): GitHub, Code Review, CI/CD Skills Integration
- Search Research (15%): Tavily, Web Research, Find Skills Merged

Overall, OpenClaw is currently better at handling tasks that are structured, highly repetitive, and have clear success criteria. Meanwhile, for users,**high-frequency, low-risk, and reversible personal workflows**are more common scenarios.**It has clear value,**saving time or attention;**it has high fault tolerance,**and errors can be reverted, redone, or taken over manually;**the interface structure is stable,**not requiring agents to operate with high precision in unfamiliar environments.

But code review automation (poor performance)

Security Issues

- **High-risk vulnerability in the native system**: OpenClaw has a large number of easily exploitable security vulnerabilities, and its underlying architecture and default configuration have inherent security flaws
    - OpenClaw has as many as 258 historical disclosure vulnerabilities, among which, out of the 82 recently exposed vulnerabilities, there are 12 ultra-critical vulnerabilities and 21 high-risk vulnerabilities, mainly of the types of command and code injection, path traversal, and access control vulnerabilities, with generally low exploitation difficulty.

- **Prompt injection and instruction overstepping risks**: OpenClaw has insufficient ability to screen and control input instructions, making it vulnerable to being induced to execute unauthorized operations
- **Sensitive Credentials and Data Leakage Risks** : Improper configuration and architectural flaws can easily lead to the leakage of core sensitive information, triggering large-scale security incidents
- **Malicious plugins account for a very high proportion**: The plugin security issues in the official skill marketplace ClawHub are prominent. Multi-round audit data shows that the proportion of malicious code in plugins reaches 10.8%-26%; an audit of 2,857 Skills found that 341 were malicious samples, mainly from the "ClawHavoc" coordinated attack campaign; an analysis of 3,016 plugins showed that 336 contained malicious code, 17.7% of plugins would obtain untrusted third-party content, and 2.9% could dynamically load external executable code
- **Imbalance between Ecosystem Capability and Security**: Compared to Anthropic Computer Use and OpenAI Operator, OpenClaw's core advantages lie in its local execution capabilities and multi-channel integration. However, this also brings about issues such as uncontrollable Skill quality and difficulty in defining the security boundaries of the open ecosystem

Behind the hype, what does OpenClaw truly represent?

Software Paradigm Shift: Evolution from Reactive Operating Systems to Agent Action Systems / For the first time, software has transformed from an "object of operation" into an "action subject".

The default assumption of past software was that humans were the sole operators of operating systems and applications. The value of software lay in providing functions, which users would gradually invoke to complete tasks. However, in systems such as OpenClaw and Claude Cowork, this assumption has been completely changed - AI can become the primary operator of software, with humans more responsible for setting goals and overseeing results. Agents can call tools across applications, combine workflows, and continuously execute tasks, making "completing tasks" rather than "providing functions" the core value of software.

- **OpenClaw** has more of an impact on the individual level, while Claude Code has more of an impact on the enterprise level.
    - This change has been verified at the market level: with the release of Agent products such as Claude Cowork, investors have begun to reprice the software industry, believing that AI can directly replace SaaS, leading to significant fluctuations in the software sector.

        - At the beginning of February, Claude Cowork launched industry plugins for sectors such as law, finance, and sales, causing SaaS stocks to collectively plunge.
        - Last Friday, Claude Code Security was released, stating that this feature can scan for security vulnerabilities in code repositories and identify software bugs for manual review. Cybersecurity stocks crashed across the board.
        - On February 24, Anthropic published a blog post on COBOL code modernization. IBM tumbled 13.2% in a single day.

- **Structurally, paradigm shift manifests as**
    - **Enterprise**: PaaS & SaaS → Training as a Service & Agent as a Service (Task Execution Capability)
        - Training as a Service
            - The Agent continuously optimizes its own behavior through the data (trajectories, feedback, rewards) generated during the runtime, transforming the training from the "offline phase" to an "online loop" and forming a closed loop of execution → feedback → learning. Whoever controls the runtime controls the training. Because: data is in the runtime, feedback is in the runtime, and the RL loop is in the runtime.
            - RL Learning Engine: Data Pipeline, Core Algorithm, Reward System
    - **Individual**: Transition from App user to personal agent commander (personal control plane)

Change in the unit of competition: Model → System stack

In the MaaS era, the focus of AI competition lies in model parameter scale and intelligence level - whoever has a larger and smarter model will win the market. In the AaaS era, the focus of competition shifts to the system-level integration capabilities of "model + runtime + memory + toolchain + human interface". OpenClaw is precisely positioned at this integration layer - it does not produce intelligence itself, but rather transforms the capabilities of various models into users' actual productivity through a standardized execution framework.

- The new competitive unit is: **model + runtime + memory + toolchain + eval/observability + human interface.**

Today, what affects the ultimate experience is no longer just how smart the model is, but also includes:

- Can runtime operate stably for a long time?

- Can memory precipitate preferences across sessions?
- Can tools and protocols be integrated into a real system?
- Can observability show why the agent fails?
- Can the interface and permission design make users willing to delegate authority?
- AWS AgentCore Runtime, LangGraph, LangSmith, and Letta correspond to runtime, memory, observability, and persistent agent memory respectively, all of which are evidence of the rise of the middle layer.

Change in economic unit: AI competition is shifting from model intelligence level to the effective output efficiency per token.

Tokens are becoming a new factor of production:

- **Token is computation**: Each Token represents a computational opportunity
- **Tokens are memory**: Longer contexts require more Tokens
- **Token is capability**: Complex tasks consume more Tokens

Economic Model Transformation:

- Shift from competition in "parameter scale" to competition in "unit token output efficiency"
- Inference efficiency (throughput/W) becomes the core indicator
- Token cost structure influences business model design

Inference efficiency becomes the core indicator:

- **Token/USD**: How many Tokens can be generated per US dollar
- **Token/Watt**: How many Tokens can be generated per watt of energy consumption
- **Task Completion Cost**: The total Token cost required to complete a specific task

Gradually formed industry stratification

> OpenClaw represents a paradigm shift from "model centrism" to "environment centrism". OpenClaw is not a product, but the birth of an Agent OS.

- **Model / Platform Layer (OpenAI / Anthropic): Developer operating system, controlling the model + Agent OS + interfaces.**

- Anthropic is making a whole-link layout for MCP, Claude Code, economic index, and agent autonomy; OpenAI is integrating computer use, agent, tools, and container; Qwen is using OpenAI-compatible API and MultiModal Machine Learning / function call / batch to open up the developer entry. What they are competing for is the default base and default protocol.

类型	玩家
Model-first OS	OpenAI、Anthropic
Platform-first OS	阿里、腾讯
Infra-first OS（补充）	AWS、云厂

Runtime Optimization - AWS's AgentCore Runtime has packaged session isolation, persistent file system, long-running, built-in auth, agent tracing, and unified SDKs for Memory / Tools / Gateway into runtime services.

- **Middle layer (the company between the model and the user control panel): The real value lies in making the Agent available - the design of the Agent runtime execution environment.**
  - Possible directions of evolution: model-agnostic harness, structured skills/plugins, file system/state abstraction, ephemeral cloud or local execution environment; Agent orchestration; workflow + memory + evaluation; evaluation & safety; observability(loop、token cost)
    - LangChain's **Deep Agents** are described as a MIT-licensed, auditable copy of the Claude Code-style agent architecture; LangGraph says memory is the core of production agents; LangSmith directly focuses on tracing, cost, latency, and failure modes;AWS AgentCore 则把 agent hosting、identity、trace、tooling 做成云 runtime。LangGraph LangChain docs.aws.amazon.com
    - **Hermes Agent** 's plugin system and its friendliness towards local models.
  - The domestic presentation form may be TaaS (also available overseas, Mechanize (Mechanize Work), Beam AI-Runtime, invisible, thinking machines lab)
    - Typical business, such as transforming an enterprise's real-world workflow into an RL training environment. It emphasizes "not just providing a model, but providing a sandbox for business scenarios".
- **Application layer: There is a possibility of the emergence of new forms - from human in the loop to human on the loop cockpit/control panel**
  - The value of Agent increasingly depends on secure execution environments, composable skills/plugins, and **workflow-native surfaces** , rather than just improvements in benchmark testing.

Conceptual Distinctions among Agent OS, Agent Runtime, and Harness Engineering

- **Agent OS: A system that manages the lifecycle of Agent actions**
  - The Environment Management Layer is responsible for coordinating user input, long-term memory, tool permissions, session state, and security isolation.
  - Core Responsibilities:
    - **Multi-terminal Access (Gateway):** Acts as the control plane;
    - **Resource Scheduling:** Manage tool sandboxing and access control;
    - **State Persistence:** Ensures that your assistant retains conversation context and task progress when switching devices (from computer to mobile phone);
    - **Infrastructure:** Solve problems in non-intelligent parts, such as network connectivity, permission allocation, and task orchestration.
  - Traditional OS manages the execution of processes and schedules resources such as CPU, memory, hard disk, peripherals, and network for them. AgentOS manages the execution of Agents and schedules**resources such as model inference, knowledge retrieval, tool invocation, file access, and cross-device collaboration**.
- **Agent Runtime: Responsible for end-to-end execution of the AI loop (Agentic Loop)**
  - Core Responsibilities
    - Orchestrate: Select the most suitable sub-agent for the current task;
    - Model Parser (模型解析): Select the most appropriate model based on task complexity (e.g., choose a cheaper or more powerful LLM);
    - Build Prompt (构建提示词): This is a dynamic process that extracts information from AGENTS.md (Roles), MEMORY (Memory), and SKILLS (Skills), and assembles it into the final instruction sent to the LLM;
    - Guard Context (Context Monitoring): Monitor Token consumption to ensure the context does not exceed the limit; Act & Repeat (Execute & Loop): Execute tool calls generated by the LLM (such as "search the web" or "write to file"), and feed the results back to the model to initiate the next round of inference.

- **Harness Engineering**: **Agent OS** and **Agent Runtime** the underlying **methodology** behind the design.**Harness repackages the capabilities of Runtime + OS into a task execution system.**

- **Codex harness—the agent loop (core loop: coordinating interactions between users, models, and tools) and logic that underlies all Codex experiences. But a complete Agent experience also encompasses more dimensions:**
  - **Thread Lifecycle and Persistence (Thread lifecycle and persistence)** - A thread (Thread) refers to a single Codex conversation between a user and an Agent. Codex is responsible for creating, restoring, forking (Forking), and archiving threads, as well as persistently storing event history to ensure that the Client can reconnect and render a consistent conversation timeline.**;**
  - **Configuration and Authentication (Config and auth)** - Codex is responsible for loading configurations, managing default settings, and running authentication processes such as "Login with ChatGPT" (including credential status management) **;**
  - **Tool Execution and Extensions (Tool execution and extensions)**Codex executes Shell or file tools in a sandbox and integrates with MCP (Model Context Protocol) services and various Skills, ensuring that they can participate in the Agent loop under a unified policy model.

- **OpenClaw 的 harness engineering**
  - Gateway: Corresponds to the "entrance standardization" of Agent OS. Gateway belongs to the outermost layer of **Agent OS**. In the past, AI was trapped in web dialog boxes. OpenClaw has achieved the integration of AI into existing social relationship chains through Gateway.
  - Built-in Skills (Skill Library): Corresponds to the "tool mounting" of Agent Runtime, which is a victory for Harness Engineering in addressing **"deterministic execution"**.
  - Memory (Memory Mechanism): Corresponds to the "state persistence" of Agent OS and belongs to the infrastructure layer of **Agent OS**. It transforms short-term context (Context) into long-term assets (Asset).
  - Heartbeat: By supplementing the "scheduling sovereignty" of the OS, Harness Engineering has achieved a victory in addressing **"Agency"**. This is the key difference between **Agent OS** and traditional software—it has the **Cron Job** attribute.
  - Soul.md / AGENTS.md (Identity Document): Corresponds to the "Personality Mount" of Runtime

Research Perspective: This is a victory for Harness Engineering in addressing "alignment and consistency". It belongs to the core input of Build Prompt in **Agent Runtime**. It is a form of **"semantic soft mounting"** for the model. Through extremely detailed Profile definitions (rather than simple System Prompts), Harness ensures that the model exhibits a stable personality across different tasks.

工程组件	Harness 的功能属性	架构层级	创造的 "Aha Moment"
Gateway	协议转换与无缝接入	**Agent OS**	“它就在我的通讯录里，像个真人。”
Skills	动作执行与环境交互	**Agent Runtime**	“它真的能帮我写文件、查资料。”
Memory	数据持久化与经验积累	**Agent OS**	“它竟然记得我上个月说过的话。”
Heartbeat	自发唤醒与主动任务	**Agent OS**	“它居然会主动提醒我该干活了。”
Soul.md	角色约束与人格塑造	**Agent Runtime**	“它说话有温度，不再是冰冷的机器。”

🎉 https://ppaolo.substack.com/p/openclaw-system-architecture-overview

OpenClaw is not a simple chatbot that wraps an AI model API. It is an AI agent operating system. OpenClaw treats AI as an infrastructure project and is committed to solving: conversation, memory, tool sandbox, access control, and task orchestration.

AI models provide intelligence; OpenClaw provides the execution environment.

OpenClaw adopts a hub-and-spoke architecture, centered around a single gateway that serves as the control plane between user inputs (WhatsApp, iMessage, Slack, macOS applications, Web UI, CLI) and AI agents:

The gateway is a WebSocket server that connects to the Notify Platform and the control interface, and distributes each routed message to the agent runtime.

Agent Runtime: Responsible for end-to-end execution of the AI loop. It assembles context from conversation history and memory, invokes models, performs tool calls against the system's available capabilities (browser automation, file operations, Canvas, scheduled jobs, etc.), and persists the updated state.

【Session Resolution、Context Assembly、Execution Loop、System Prompt Architecture】

OpenClaw's core advantage lies in its separation of the interface layer (message source layer) from the assistant runtime (the layer where intelligence and execution reside). This means that a persistent assistant can be accessed via any used messaging application, with conversation state and tool access both centrally managed by the user's hardware.

Other References

- OpenClaw Pi Agent: Efficient Agentic Loop, decoupled design based on RPC **(Remote Procedure Call)** , sophisticated context management mechanism, and personalized workspace
    - **Agentic Loop**
        - **Understanding Intent** : Receive task instructions and relevant context from the Gateway.
        - **Prompt Construction:** Dynamically construct a structured and informative prompt for interacting with large language models (LLMs).
        - **LLM Interaction**: Send the constructed prompt to the configured LLM (such as OpenAI's GPT series or Anthropic's Claude series) and wait for its response.
        - **Response Parsing:** Parse the response returned by the LLM. This response may be a direct text answering the user or an instruction requiring the invocation of a tool.
        - **Taking Action**: If the response is an instruction to invoke a tool, Agent Runtime will execute the corresponding tool via Gateway, and use the execution result as new Observation information to enter the loop again; if the response is the final answer, it will be returned to Gateway, which will then send it to the user.
- Context Management: The Key to AI Memory

An excellent AI assistant must have good memory capabilities and be able to understand the context of conversations. Agent Runtime has done a great deal of meticulous work in this regard. It maintains a separate **context window** for each session.

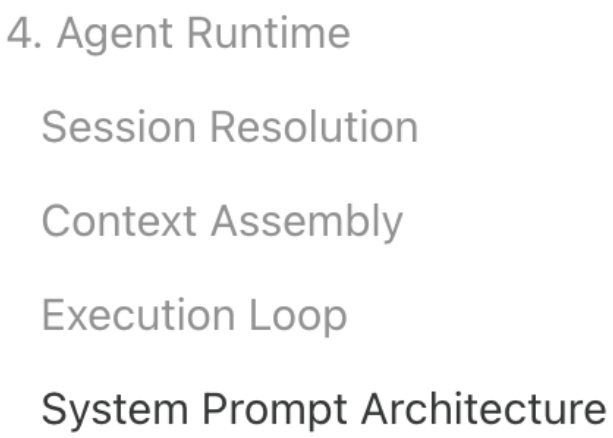
Before each interaction with the LLM, the Agent Runtime carefully prepares context information, which typically includes:

- **System Prompt**: The basic role setting and code of conduct given to the AI.
- **Message History**: Recent conversation history to help the AI understand the current context.
- **Available Tools (Available Tools)**: Informs the AI about which tools it can use in the current environment and how to use them.
- **Skills Knowledge (Skills Knowledge)**: Inject relevant skills information to guide AI in completing tasks in specific domains**.**

When the conversation history is too long, to avoid exceeding the context length limit of the LLM, Agent Runtime will also intelligently **compact or summarize** the historical messages, refining them to retain only the most important information.

- Workspace: Agent's personalized base

To ensure that each Agent instance can have its own independent configuration, skills, and memory, OpenClaw introduced the concept of **Workspace**. By default, each Agent's workspace is located in the `~/.openclaw/workspace` directory. In



this directory, personalized instruction files can be configured for the Agent, for example:

- `AGENTS.md: Defines the core behaviors and roles of`
- `SOUL.md: Inject deeper "soul" or personalized`
- `TOOLS.md: Defines the specific set of tools available to this Agent.`

Trend Outlook

What is certain to continue?

Further diffusion of technology, but the path is to first transform people into stronger managers in specific scenarios, and then gradually automate some processes

- Anthropic data shows that in the US workplace, AI adoption has doubled from 20% to 40% in just two years, a pace far exceeding that of transformative technologies such as electricity (30+ years), personal computers (20 years), and the early internet (5 years). However, behind this rapid spread lies a significant imbalance. This imbalance not only follows the traditional pattern of technology diffusion (initially concentrated in a few nodes) but is also more extreme in the AI era.A notable feature of early technology adoption is its *concentration* — both geographically limited to a few regions and confined to a few tasks within enterprises. As we describe in this report, the application of artificial intelligence in the 21st century also seems to follow a similar pattern, although its development cycle is shorter and its intensity is higher than that of technological diffusion in the 20th century.
- Agent is currently at a critical juncture in the transition from "early adopters" to "early majority". Anthropic reports show that although programming remains the most common use, tasks related to computer and mathematics occupations account for 35% of Claude.ai conversations, but Claude's application scope has expanded to low-wage tasks. With the diversification of usage scenarios, the average economic value of work completed on Claude (measured by the wage levels of relevant occupations in the US) has slightly declined.This model conforms to the standard "adoption curve" theory, which posits that early adopters tend to engage in specific high-value uses, while later adopters take on a broader range of tasks.

- Anthropic's report shows that AI usage still mainly focuses on augmentation, with a proportion of approximately **57% vs 43%**; in subsequent reports, as tasks such as coding are migrated to APIs, the tendency towards automation increases. This means that the most realistic path in the next few years is not "digital employees fully replacing humans", but rather **first transforming humans into stronger managers and then gradually automating some processes**. Anthropic Anthropic

Personal Agent will continue to develop, especially in the directions of edge-side, MultiModal Machine Learning, and personalization

- Android explicitly defines Gemini Nano as a privacy-first, low-cost, and offline-capable on-device AI; Apple has also opened up the Foundation Models framework. Whoever controls user devices, system entry points, sensors, and local context is more likely to create a truly sustainable personal agent. developer.android.com www.apple.com

New integrated hardware / Hardware system centered around individuals and families

- If AI truly becomes a permanent resident in mobile phones, computers, headphones, and smart home nodes, then the future will resemble more of a "cross-device ambient intelligence" rather than an isolated AI terminal. Edge models, local permissions, and cross-device context will be more important than the form factor of a single piece of hardware. The underlying basis for this judgment is still that both Apple and Android are directly integrating AI into their operating systems and development frameworks.

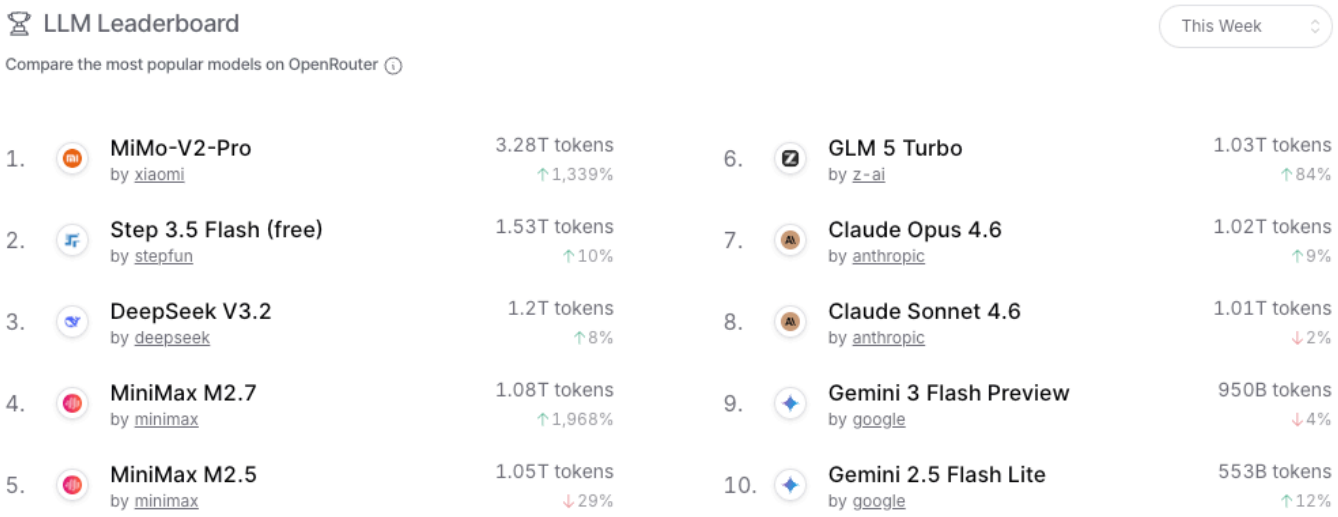
Vision: The middle layer may potentially penetrate downward into the production environment and directly become the future/vertical domain control panel for users.

- The harness itself needs to be bound and adapted to specific business scenarios, **how to design a good human in the loop cycle, and gradually transition to human on the loop in the future as the model's capabilities develop - Agent action system design** : From human in the loop to human on the loop: Design a governable system that transforms model intelligence into actionable capabilities. The current practice is harness engineering, which addresses robustness and user operability.
  - Training as a Service（vs Token as a Service，偏 infra）
  - **Control panel：从 human in the loop 到 human on the loop**

        - As users become more familiar with agents, they will be more willing to auto-approve, but also more willing to interrupt at critical moments; and models voluntarily stopping to seek clarification is more common than humans actively interrupting. The future human-machine relationship is not "humans gradually relinquishing control", but rather**humans transitioning from gradual approvers to supervisors**.Anthropic
    - A large number of middle-tier companies have emerged in areas such as Agent orchestration, workflow, memory, and evaluation. (See Appendix)
        - LangChain's deep agent, hermes agent, AutoGen's multi-agent framework, and Rivet's drag-and-drop GUI workflow builder are all lowering the threshold for Agent development.
        - Memory is not chat history, but rather long-term preferences, task context, rules, and reflection mechanisms. LangGraph divides memory into short-term and long-term, and Letta even turns agent memory into a git-backed context repository, and conducts memory organization during the sleep period through reflection/subagent. This type of ability determines whether a personal agent will change from "getting to know you anew every time" to "becoming more and more like you the more you use it".
        - Evaluation (worth noting). The LangChain report shows that 52% of respondents have adopted evaluation (evals) solutions, while 89% have deployed observability. Observability has preceded evaluation as the primary investment in Agent engineering, reflecting the industry's strong demand for "knowing what the Agent is doing".
        - Sandbox (maybe a big company opportunity)
        - Safety (opportunity)
        - Workflow orchestration/durable execution (Multi-agent orchestration opportunities open)hermes agent

China's Advantages

- Token usage of Chinese models accounts for half of the total, indicating that there is still a large amount of demand that can be activated



    - Compared with overseas cutting-edge models, Chinese models still have significant room for price increases, which also provides fertile ground for early exploration opportunities
    - When cost, openness, and fine-tunability are more user-friendly, the call volume will be quickly released.

- Advantages: energy costs, infrastructure/location, and a huge market volume.
- Shortcoming: The gap in advanced tools and computing power, as well as system scale and integration, results in higher unit computing power resource costs.

What could be the noise?

- Single-purpose Agent hardware terminal: Agent dedicated computer
    - The underlying logic behind the existence of such products is the lack of a sense of security in human control over AI behavior. Once the issues of security and visualization are resolved and more suitable usage methods are developed, such products are likely to become redundant devices, i.e., they lack competitiveness in a single purpose and do not have multiple uses.
    - On-device intelligence will ultimately become a native capability of mobile phones and operating systems. So the long-term issue with many single-purpose AI terminals is: **they lack both the versatility of mobile phones and the low-cost support of cloud services**. So are they more like transitional products?
- Digital employees: fundamentally, the issue of robustness needs to be addressed

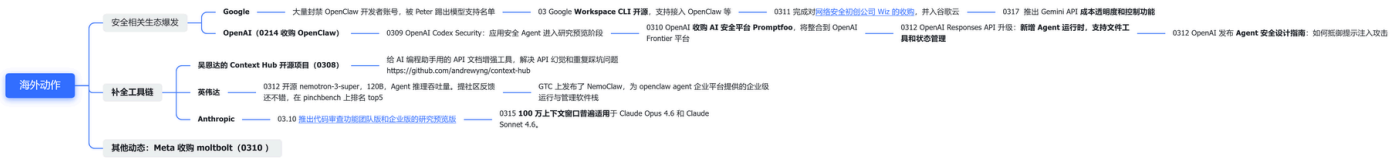
- Despite the previously mentioned improvements in model capabilities, especially in computer use, the results from OSWorld and WebArena indicate that the stable execution of complex open-ended tasks remains fragile; Anthropic's real-world data shows that most tool calls still involve human-in-the-loop. What is more likely to happen is not that enterprises will hire 10,000 AI employees overnight, but rather **that enterprises will first agentize a portion of repetitive, low-risk, and auditable processes** .
- Similarly, referring to the development of past industrial revolutions, productivity first declines because of the initial adaptation between humans and new technologies (a new study on user behavior published by Anthropic), from full automation to AI augmentation (increasing task complexity and deep-water scenario testing). The singularity has arrived, so why hasn't the economy experienced explosive growth?
  - **Brynjolfsson argues (2017) that the time lag explanation is the most convincing and the main cause of the productivity paradox** . Paul David points out (2000): "The early stages of paradigm transformation should not be expected to yield the maximum productivity returns, although the diffusion rate of new technologies may be the fastest at this time." General-purpose technologies need to undergo multiple secondary innovations, complementary innovations, and organizational changes before they can have a substantial impact on productivity. Brynjolfsson (2020) summarizes the lag effect of general-purpose technologies on productivity as the "J-curve". Helpman and Trajtenberg (1994) hold a similar view, dividing the impact of general-purpose technologies on economic growth into two stages: sowing and harvesting. During the sowing stage, productivity growth is slow or even declines, and only in the harvesting stage does growth truly begin.

Problems to be solved to achieve the above trends

- **Model Layer**
  - MultiModal Machine Learning + Architecture innovation optimized around inference efficiency + Enhancement of long context capabilities & Self-evolution (training the harness-completion ability into the model)
    - MultiModal Machine Learning (the key from terminal use, web use to real computer use);
      - **MultiModal Machine Learning, especially video, is compute-bound, and sparse/similarity utilization will become a key technical approach**
        - Text inference is memory-bound; video generation, with the addition of diffusion/transformer, has a computational load 2–3 orders of magnitude higher and becomes compute-bound.

        - Draw on traditional video compression: shift from "compressing transmission volume" to "compressing computational volume", with the core being to leverage intra-frame/inter-frame token similarity for grouping and sparse computation.
    - Continuous Improvement of Model Capabilities: Long Context, Self-Evolution
        - Currently, the leading edge generally reaches a million-context window, but in actual use, it is not that large
        - Self-evolution: OpenAI, Claude, MiniMax, etc. have all recently mentioned it
    - Architecture innovation centered around optimizing inference efficiency
        - Further sparsification
        - Innovation of residual attention mechanism similar to Kimi
    - Specific model optimized for inference scenarios
        - Edge + MultiModal Machine Learning
- **System and Infra Layer: The industrial paradigm is shifting from "training supremacy" to "inference and task efficiency supremacy", and the main battlefield for improving inference efficiency is shifting from the token layer to the "task layer".**
    - **Software-Hardware Collaboration (Algorithm + System + Hardware)**
    - Continuously reduce**single-task cost**through smaller models/fewer invocation times, fewer tool invocations, and better planning and verification mechanisms.
    - Specialization of inference chips; an interesting direction - etching model weights into the chip
    - To support "real-time interactive content/streaming generation on mobile devices", without a jump in energy density, it must rely on **an energy efficiency improvement of 1–2 orders of magnitude or more** .
        - Two major challenges for the terminal in the future:
            1. Redesign of Sensor/Input/Output (Edge-side MultiModal Machine Learning Input/Output System)
            2. Edge Computing and Energy Efficiency (Underlying Chip/Packaging/Storage Close to Logic)

Appendix: Overseas Actions



Big Factory

Google

- A large number of OpenClaw developer accounts were banned and kicked out of the model support list by Peter
- 03 Google **Workspace CLI Open Source** , Supports Integration with OpenClaw, etc.
- On 0311, completed the acquisition of cybersecurity startup Wiz and integrated it into Google Cloud
- 0317 Launches Gemini API **Cost Transparency and Control Features**

OpenAI (Acquired OpenClaw in 0214)

- 0309 OpenAI Codex Security: Application Security Agent Enters Research Preview Phase
- 0310 OpenAI **acquires AI security platform Promptfoo**, to be integrated into the OpenAI Frontier platform
- 0312 OpenAI Responses API Upgrade:**New Agent Runtime Added, Supporting File Tools and State Management**
- 0312 OpenAI Releases **Agent Security Design Guide**: How to Defend Against Prompt Injection Attacks

Anthropic

- 03.10 Released the Research Preview of Code Review Feature Team Edition and Enterprise Edition
- 0315 **1 million context window is generally applicable** to Claude Opus 4.6 and Claude Sonnet 4.6.

bonus

Wu Enda's Context Hub Open Source Project (0308)

API documentation enhancement tool for AI programming assistants, addressing API hallucination and repeated pitfalls https://github.com/andrewyng/context-hub

NVIDIA

- 0312 Open Source nemotron-3-super, 120B, Agent Inference Throughput. Community feedback is quite good, ranking in the top 5 on pinchbench

- NemoClaw, an enterprise-level runtime and management software stack for the openclaw agent enterprise platform, was released at GTC

Meta acquires moltbolt (0310)

Startup

Crunchbase data shows that from 2024 to 2025, the AI Agent toolchain will become a core track for venture capital, with financing in the four major core sub-directions continuing to explode. In 2025, the total global financing in this field will exceed $10 billion, nearly tripling compared to 2024.

Memory (Agent Memory Infrastructure)

- Overall Scale: In 2024, the financing amount in the track exceeded $120 million, and in 2025 it doubled to over $250 million, making it one of the fastest-growing directions in Agent infrastructure.
- Core target: The leader in the field Mem0 has cumulatively completed $24 million in financing, with a post-investment valuation of $150 million, positioning itself as a general memory layer and being compatible with mainstream Agent frameworks such as OpenAI, Claude, and LangGraph; Letta (formerly MemGPT): A benchmark for open-source Agent long-context memory systems, developed by the UC Berkeley team, pioneered the hierarchical memory architecture, with over 30,000 GitHub stars, has cumulatively completed over $15 million in financing, focuses on infinite context persistent memory and autonomous memory management, and is deeply adapted to multi-agent collaborative scenarios
- Capital Hotspots: Focusing on directions such as cross-session persistent memory, long context optimization, enterprise-level multi-tenant isolation, and memory security auditing, open-source memory projects like claude-mem and cognee quickly brought in new users at the beginning of 2026, with their GitHub star counts skyrocketing, becoming new focal points for capital

Security (Agent Native Security)

- Overall Scale: From 2024 to 2025, cumulative financing in the Agent security sub-segment exceeded $800 million, and cumulative seed-round financing in 2025 exceeded $70 million, making it the sub-segment with the highest capital attention in Agent infrastructure

According to Crunchbase data, investors have invested more than **$9 billion** in seed-stage financing of global AI startups over the past six months.

Key Focus Areas:

- **Cybersecurity**: Oasis Security Secures $120 Million in Financing, Focusing on AI Agent Identity Security
- **Multimedia AI**: Creative tools such as video generation and image editing
- **Robotics** : Embodied Intelligence, Industrial Robots
- **Work Automation**: Process Automation, Document Processing

AI security startups' financing reached **$6.34 billion** in 2025, nearly tripling from $2.16 billion in 2024. Software Strategies Blog7

- Core target:
  - Oasis Security: Completed $120 million in financing in 2025, focusing on AI Agent identity security, and is the largest single early-stage financing in this track
  - Mind Network: Has cumulatively completed $12.5 million in seed + Pre-A round financing, focusing on Agent encrypted operation and whole-link security protection
  - Kai: Completed a combined Seed + Series A financing of $125 million in 2026, setting a new record for early-stage financing in the AI security field, focusing on AI-native automated defense and deeply adapted to Agent scenarios
- Capital Hotspots: Prompt Injection Defense, Memory Pollution Protection, Plugin Security Audit, Agent Behavior Auditable, Zero-Trust Permission Management and Control, etc.

Environment (agent RL simulation environment)

- Overall Scale: Cumulative financing from 2024 to 2025 will exceed $1.2 billion, making it the core infrastructure track for Agent implementation, with large-scale financing of Series B and above exceeding $600 million in 2025
- Core target:
  - OpenClaw Ecosystem: By March 2026, there were over 4,800 OpenClaw-based startups globally, with over 1,200 having completed financing, total financing exceeding $8.7 billion, and the average valuation of leading projects increasing by 400% within 6 months.
  - Anysphere (developer of Cursor): After completing multiple rounds of financing in 2025, its valuation reached $29.3 billion, with a focus on the programming Agent operating environment
  - Daytona: A manufacturer of composable runtime environments for AI Agents, has cumulatively completed $31 million in financing (including $24 million in Series A), led by FirstMark; features an Agent-specific sandbox that can complete resource provisioning within 60 milliseconds, supports full control of the operating system and computing resources by Agents, and is compatible with the entire workflow of code execution and computer operations

    - E2B: The leader in the dedicated cloud sandbox track for Agents, completed a $21 million Series A financing in July 2025, led by Insight Partners; its core product is an open-source lightweight Agent security sandbox, with 88% of Fortune 100 companies already using its infrastructure, and customers covering leading AI manufacturers such as Hugging Face, Perplexity, and Groq
    - Blaxel: A cloud infrastructure provider for ultra-fast agents, specializing in lightweight agent runtime sandboxes that can be launched within 3 seconds, compatible with mainstream agent protocols, and providing secure and isolated runtime environments for scenarios such as code execution and web operations
- Capital Hotspots: Local Execution Sandbox, Isolated Runtime Environment, Multi-terminal Adaptation Framework, Simulation Test Environment, Industrial/Embodied Agent Real-time Runtime Engine, etc.

Evaluation (Agent Evaluation System)

- Overall Scale: Cumulative financing from 2024 to 2025 exceeds $320 million, with financing in 2025 growing by over 200% year-on-year, making it a track with inelastic demand for the implementation of enterprise-level Agents 36Kr
- Core targets: The track is mainly composed of vertical startups, and in the first half of 2025, 3 Agent evaluation startups completed seed round financing in the tens of millions of dollars;Arthur AI: A leading player in the Agent evaluation track, has cumulatively completed over $60 million in financing, focusing on Agent capability benchmark testing, hallucination detection, and robustness evaluation, serving over 300 large enterprises. Kolena: An enterprise-level Agent evaluation platform, will complete $15 million in Series A financing in 2025, focusing on quantitative evaluation of multi-agent collaborative task completion rate and industry adaptability assessment. Patronus AI: An AI security and evaluation vendor, has cumulatively completed $29 million in financing, focusing on Agent compliance auditing and prompt injection risk evaluation, suitable for highly regulated scenarios such as finance and government affairs.
- Capital Hotspots: Agent Capability Benchmarking, Robustness Assessment, Security Compliance Audit, Hallucination Detection, Task Completion Rate Quantitative Evaluation, Industry Adaptability Assessment, etc.

Multi-Agent Orchestration

- Overall Scale: From 2024 to 2025, global cumulative financing will exceed $4.8 billion, making it the core infrastructure track for AI Agent to move from pilot to large-scale implementation. In 2025, large-scale financing of Series B and above for enterprise-level commercial platforms will exceed $2.2 billion.

- Core Target: LangChain Technologies: The de facto global standard for multi-agent orchestration, with a developer market share exceeding 60%, over 100,000 cumulative stars on GitHub, and cumulative financing exceeding $60 million, with participation from top-tier institutions such as Sequoia and Benchmark;CrewAI Labs: Pioneering the role-playing multi-agent collaborative paradigm, with over 20,000 GitHub stars, completed $12 million Series A financing in 2025, post-investment valuation of $180 million, and enterprise edition customer growth rate exceeding 400%
- Capital Hotspots: General Distributed Orchestration Engine, Domestic Independent and Controllable Orchestration Framework, Visual Zero-Code Orchestration Platform, Vertical Industry End-to-End Process Orchestration Solution, Multi-Agent Observability and Governance Tools, Cross-Platform Agent Communication Protocol, etc.

年份	美国 AI 安全初创企业总融资额	同比增速	核心市场特征
2024 年	21.6 亿美元	-	全年 49 家 AI 相关初创公司完成单轮 1 亿美元及以上融资，AI 安全是核心细分赛道；头部项目 Abnormal Security 完成 2.5 亿美元 D 轮融资，投后估值 51 亿美元
2025 年	63.4 亿美元	193.5%（近 3 倍）	仅上半年就有 25 家 AI 安全相关初创公司完成超 1 亿美元融资；全球 AI 安全融资中，美国市场占比超 65%，旧金山湾区为核心聚集地，2025 年 Q2 单季度该区域相关融资超 550 亿美元
2026 年（截至 3 月）	超 30 亿美元（不含大额并购）	延续高增长	仅前两个月，美国 AI 领域总融资超 250 亿美元，AI 安全占比超 12%；Shield AI 完成 20 亿美元 G 轮融资，Safe Superintelligence 正在筹集超 10 亿美元融资，聚焦 AGI 安全

Other

Tooling & Auth: Composio / Arcade. Although they also handle orchestration, their greatest value lies in providing over 500 authorized software interfaces (Slack, GitHub, Salesforce) for Agents and managing Agents' authentication and state memory when using these tools.